RippleLab Day 01 Progress Report

Day: 01 of 30

Date: 2026-08-12

Status: Complete locally; GitHub publication blocked

Branch: `codex/day-01-bootstrap`

Implementation commit: `9032622`

GitHub remote: Not configured

CI: Workflow created; remote run unavailable

1. Objective

Establish a runnable, documented and verifiable foundation for RippleLab. Day 1 covers the monorepo, web and API entrypoints, shared contracts, environment safety, product scope, architecture, user journeys, automated checks and the initial Git checkpoint.

2. Acceptance Criteria

- [x] The Next.js web app starts locally and responds at `http://localhost:3000`.
- [x] The FastAPI service starts locally and responds at `http://localhost:8000`.
- [x] One root command, `pnpm check`, runs all core Day 1 checks.
- [x] The repository contains a safe `.env.example` and no real credentials.
- [x] A GitHub Actions CI skeleton is present.
- [x] Product boundaries, user journeys and the non-advice disclaimer are documented.
- [x] The implementation is committed on a focused Day 1 branch.
- [ ] The branch is pushed to GitHub. No remote is configured and both available gh account tokens are invalid.

The build and local verification criteria are complete. GitHub publication is the only unmet external criterion.

3. Completed Work

Runnable application foundation

- Created a Next.js 16.3 App Router application with TypeScript, Tailwind CSS 4 and ESLint.
- Replaced the generic scaffold with a RippleLab landing surface that states the five scenario families and educational-use boundary.
- Created a separately packaged FastAPI economic service with root and liveness endpoints.
- Added a closed JSON Schema and TypeScript type for the API health response.
- Locked JavaScript and Python dependency graphs with `pnpm-lock.yaml` and `uv.lock`.

Product and engineering contract

- Defined the target user, product promise, five supported scenarios, MVP capabilities, non-goals and trust requirements.
- Documented four primary user journeys plus failure journeys.
- Recorded the system context, data flow, repository boundaries and planned deployment.
- Accepted ADR 0001: pnpm monorepo plus deterministic Python calculation engine.
- Established the rule that LLMs may parse or explain structured data but cannot calculate authoritative financial outputs.

Repository quality and safety

- Added root scripts for development, production build and a single-command quality gate.
- Added ESLint, TypeScript, Ruff, pytest, contract and common-secret checks.
- Added GitHub Actions jobs for the web/contracts and economic engine.
- Added environment-variable documentation with blank or local-only values.
- Added ignore rules for dependencies, virtual environments, builds, local environments, caches and OS files.
- Added a changelog and evidence-registry placeholder.

4. Technical Implementation

The repository is a pnpm workspace with three current projects: the root orchestrator, `@ripplelab/web` and `@ripplelab/contracts`. The Python service is managed independently by uv so its runtime and deployment can remain isolated from the web app.

The API version is `0.1.0`. `GET /health` returns only `status`, `service` and `version`, preventing accidental environment disclosure. Pydantic validates the response while a matching JSON Schema establishes the future cross-language contract boundary.

The web app uses the App Router under `apps/web/src/app`. Its root page is statically prerendered. The production check uses Next.js's supported webpack option because the managed sandbox prevents Turbopack's CSS worker from binding a local port during `next build`; local development was independently verified with the supported webpack development mode.

5. Calculations, Assumptions and Evidence

No financial formulas or economic assumptions were implemented on Day 1. Deterministic loan and deposit primitives begin on Day 6.

The evidence directory now defines the metadata expected for future sources: publisher, title, URL, publication/effective date, access date, supported claim and dependent model parameters.

6. Verification Evidence

Check	Command or method	Result
Shared contract	`pnpm check:contracts`	Passed
Web lint	ESLint through `pnpm check:web`	Passed
Web type-check	`tsc --noEmit`	Passed
Production web build	Next.js 16.3 webpack build	Passed; `/` statically prerendered
API lint	Ruff check	Passed
API formatting	Ruff format check	Passed; 4 files formatted
API tests	pytest	Passed; 2 tests
Secret scan	`pnpm check:secrets`	Passed
Full quality gate	`pnpm check`	Passed
API smoke test	HTTP GET `/` and `/health`	Passed; both returned HTTP 200
Web smoke test	HTTP GET `/`	Passed; returned HTTP 200 and expected RippleLab content
Git whitespace check	`git diff --check`	Passed

Pytest emitted one upstream deprecation warning from FastAPI/Starlette's test client compatibility layer. It does not fail the tests or affect the API response. It should be revisited when the upstream testing stack finalizes its replacement client.

7. Important Files Changed

- `README.md` - product boundary, local setup, quality checks and repository guide.
- `package.json` and `pnpm-workspace.yaml` - root workspace and one-command orchestration.
- `apps/web/src/app/page.tsx` - initial RippleLab landing surface.
- `services/economic-engine/src/ripplelab_engine/main.py` - FastAPI entrypoint and health contract.
- `services/economic-engine/tests/test_health.py` - HTTP contract and non-disclosure tests.
- `packages/contracts/schemas/health-response.schema.json` - canonical Day 1 schema.
- `.github/workflows/ci.yml` - GitHub Actions checks.
- `.env.example`, `.gitignore` and `scripts/check-secrets.mjs` - credential and repository safety.
- `docs/PRODUCT_CONTRACT.md`, `docs/USER_JOURNEYS.md` and `docs/ARCHITECTURE.md` - product/technical contract.
- `docs/decisions/0001-monorepo-and-deterministic-engine.md` - architectural decision record.

The implementation checkpoint contains 51 files and 6,522 inserted lines, including generated lockfiles and the previously approved planning documents.

8. Visual Proof

The product surface was verified by its rendered HTTP response and production prerender. A UI screenshot is not included because Day 2 owns the visual design system and responsive application shell.

9. Security, Privacy and Accessibility

- No real credentials or personal financial data are present.
- Sensitive server values are not exposed through `NEXT_PUBLIC_` names.
- The API liveness response contains no environment or infrastructure details.
- CI permissions are read-only for repository contents.
- The landing page uses semantic headings, sections and readable high-contrast colors.
- Formal accessibility component testing begins on Day 2.

10. Decisions

- Use a pnpm monorepo so web and contracts share a lockfile and commands.
- Keep FastAPI under uv so deterministic calculations retain Python-native testing and simulation tooling.
- Treat JSON Schema as the canonical cross-language contract boundary.
- Keep the LLM outside authoritative numeric calculations.
- Keep `main` deployable and use short-lived `codex/day-XX-topic` branches.
- Use webpack for managed-environment builds until Turbopack can create its required local worker process; this is a supported Next.js mode.

11. Risks and Blockers

- **GitHub publication blocked:** The repository has no `origin` remote. `gh auth status` also reports invalid tokens for `nayaksiddartha397` and `Siddarthnayak18`. Resolution requires re-authentication and either an existing repository URL or an explicit repository creation choice, including visibility.
- **Dependency freshness:** Versions are locked for reproducibility but should be upgraded intentionally with tests, not automatically during daily builds.
- **Cross-language drift:** Only the health contract exists today. Day 5 must establish generation or stronger contract tests before simulation schemas expand.
- **Upstream test warning:** FastAPI's current test client emits a Starlette deprecation warning. Monitor and migrate when upstream guidance stabilizes.

12. GitHub Publication

- Implementation commit message: `chore(repo): bootstrap RippleLab monorepo`
- Implementation commit SHA: `9032622`
- Local branch: `codex/day-01-bootstrap`
- Remote branch: Not pushed
- Pull request: Not created
- CI URL/status: Not available; workflow exists locally

No push or CI success is claimed.

13. Next Day

Day 2 will establish the RippleLab design system and responsive application shell: visual tokens, typography, navigation, dashboard shell, reusable controls, loading/empty/error states and accessibility smoke coverage.

Prerequisite for the publication workflow: configure a valid GitHub login and an `origin` repository.

Sign-off

Prepared by: Codex

Evidence reviewed: Yes

Ready to continue: Yes, after recording the publication blocker